Exercise IV, Algorithms 2025-2026

These exercises are for your own benefit. Feel free to collaborate and share your answers with other students. There are many problems on this set, solve as many as you can and ask for help if you get stuck for too long. Problems marked * are more difficult but also more fun :).

Heaps and Heapsort

1 Illustrate the steps of HEAPSORT by filling in the arrays and the tree representations on the next page.

2 (Exercise 6.1-4 in the book) Where in a max-heap might the smallest element reside, assuming that all elements are distinct?

3 (half a *, Problem 6-1 in the book) We can build a heap by repeatedly calling MAX-HEAP-INSERT to insert the elements into the heap. Consider the following variation on the BUILD-MAX-HEAP procedure:

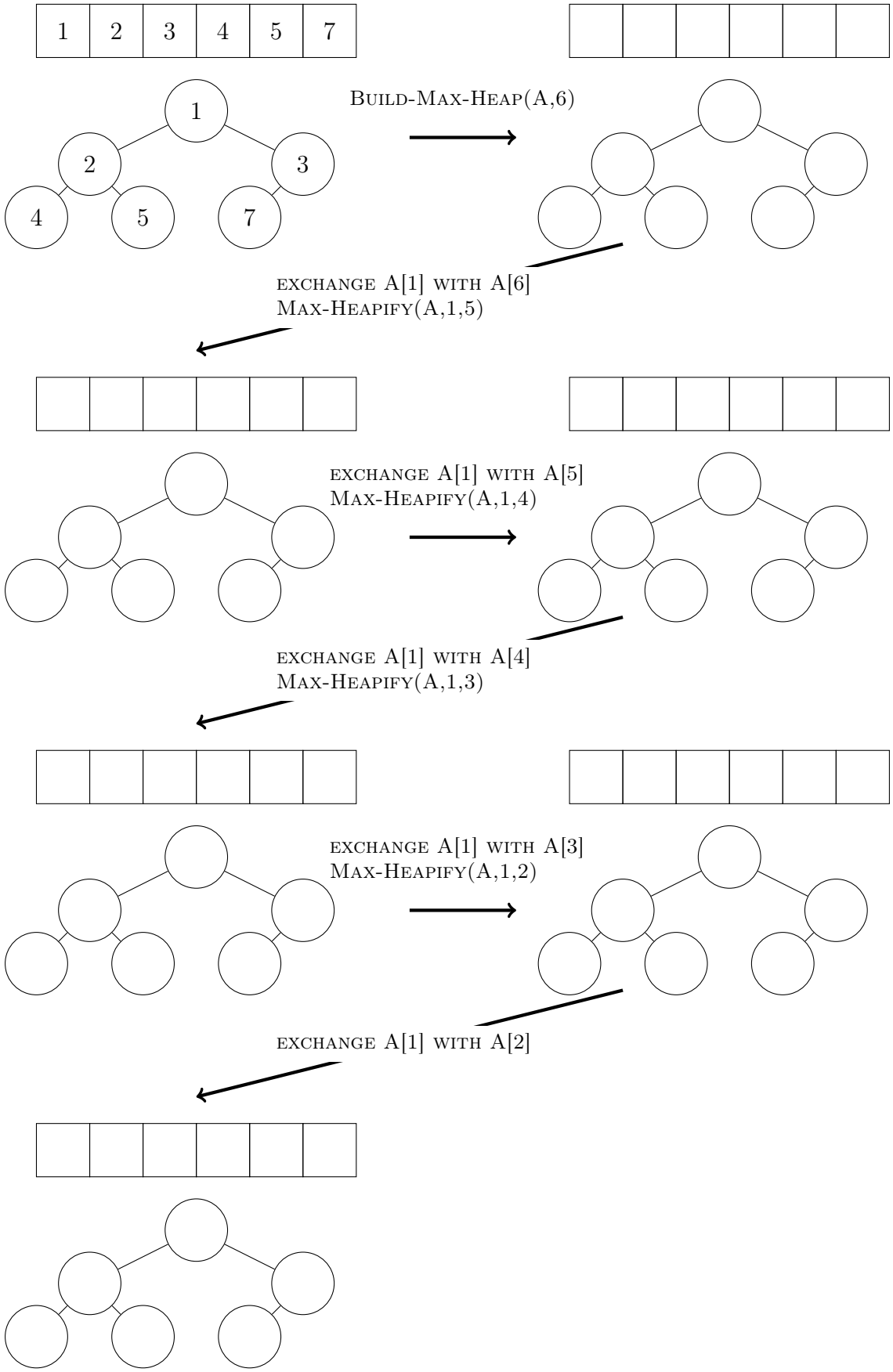
```
BUILD-MAX-HEAP'(A)
A.heap-size = 1
for i = 2 to A.length
    MAX-HEAP-INSERT(A, A[i])
```

3a Do the procedures BUILD-MAX-HEAP and BUILD-MAX-HEAP' always create the same heap when run on the same input array? Prove that they do, or provide a counter example.

3b Show that in the worst case, BUILD-MAX-HEAP' requires $\Theta(n \lg n)$ time to build an n-element heap (in comparison to BUILD-MAX-HEAP that we saw in class only needs time $\Theta(n)$).

4 (*, Exercise 6.5-9 in the book) Give an $O(n \lg k)$-time algorithm to merge k sorted arrays into one sorted array, where n is the total number of elements in all the input arrays.

Hint: Use a min-heap of size k for k-way merging.


1 2 3 4 5 7
1
2
3
4
5
7
Build-Max-Heap(A,6)
exchange A[1] with A[6]
Max-Heapify(A,1,5)
exchange A[1] with A[5]
Max-Heapify(A,1,4)
exchange A[1] with A[4]
Max-Heapify(A,1,3)
exchange A[1] with A[3]
Max-Heapify(A,1,2)
exchange A[1] with A[2]

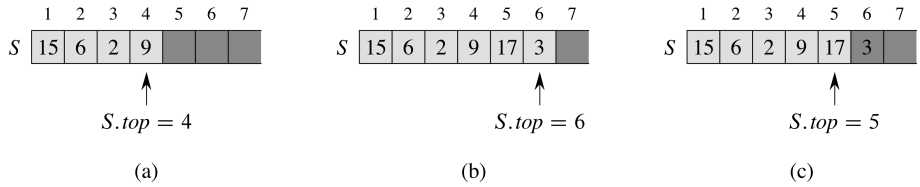
5 *(*, previous exam question)* Consider the following problem:

> **INPUT:** A positive integer k and an array $A[1 \ldots n]$ consisting of $n \geq k$ integers that satisfy the max-heap property, i.e., A is a max-heap.
>
> **OUTPUT:** An array $B[1 \ldots k]$ consisting of the k largest integers of A sorted in non-decreasing order.

Design and **analyze** an efficient algorithm for the above problem. Ideally your algorithm should run in time $O(k \log k)$ but the worse running time of $O(\min\{k \log n, k^2\})$ is also acceptable.

Queues, Stacks, Lists

6 (Exercise 10.1-1 in the book) Using Figure 10.1 in the book (see below) as a model, illustrate the result of each operation in the sequence PUSH(S,4), PUSH(S,1), PUSH(S,3), POP(S), PUSH(S,8), and POP(S) starting from the stack depicted in (a).



(a) (b) (c)

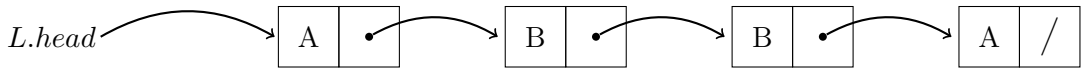
7 (Exercise 10.1-2 in the book) Explain how to implement two stacks in one array $A[1 \ldots n]$ in such a way that neither stack overflows unless the total number of elements in both stacks together is n. The PUSH and POP operations should run in $O(1)$ time.

8 (Exercises 10.2-2 and 10.2-3 in the book)

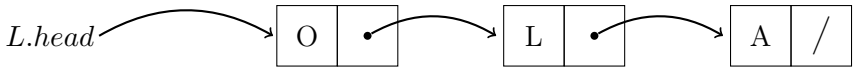
8a Show how to implement a stack by a singly linked list (the operations PUSH and POP should still take $O(1)$ time).

8b Show how to implement a queue by a singly linked list (the operations ENQUEUE and DEQUEUE should still take $O(1)$ time).

9 *(15pts)* **Palindrome.** A word is a palindrome if its reverse is equal to itself. For example, ABBA is a palindrome whereas OLA is not. One way of representing a word in a computer is to have a single-linked list where we have a list-element for each letter. For example, ABBA is represented by the single-linked list



and OLA is represented by the single-linked list



Design and **analyze** an algorithm that, given a pointer to the head of a single-linked list which represents a word, outputs YES if the word is a palindrome and NO otherwise.

For full score your algorithm should run in **linear time** and **should not use any other data structures than single-linked lists**, i.e., no arrays, stacks, queues, etc..

10 (*, Exercise 10.2-7 in the book) Give a $\Theta(n)$-time nonrecursive procedure that reverses a singly linked list of n elements. The procedure should use no more than constant storage beyond that needed for the list itself.